

Button — Component Spec Sheet

Oct 1, 2026 · @Anmol

Overview

The primary button is a 52pt-tall dark button with 16pt corners, a 2pt gradient border and two inner shadows. At rest it looks *raised*. Under the finger it sinks *pressed in*. The visual design follows the Figma frame [Button design](#) (node 1088:18515). The iOS reference build is `DomeButtonStyle` in `GyroQR/SlabDome.swift`.

It comes in two variants. They share one press, one motion and one haptic.

Variant	Figma	Use	Label colour
dark (default)	M-PrimaryButton (H52)	Primary action: Yes, use this email; Continue	#FFFFFF
light	M-SecondaryButton (H52)	Secondary action beside a primary: Edit email, Back	#1D2539

Rest state = the Figma design, exactly. Nothing is added at rest: no drop shadow, no label shadow. **Pressed state = designed here**, because Figma has none. On press the button reads as **pushed down into the surface**. The edge of the surface shades its top, a faint light bounces up from the bottom, and it drops 0.5pt while shrinking to 0.98. The light change takes **20ms**.

Anatomy and layout

The button has a 52pt fixed height and 16pt corners. It is full width inside its container: 327pt inside the 351pt action-bar card. The shadow sizes below are in units of the **half-height h = 26pt**, so they scale if the height ever changes.

Property	Value
Size	Full width × 52pt (327 × 52pt in the design)
Corner radius	16pt, continuous corners (Figma corner smoothing 100% on primary, 60% on secondary)
Border	2pt, inside the bounds

Property	Value
Label	Single line, centred, optional 24pt leading icon with an 8pt gap
Hit area	The rounded rect itself, 52pt tall (above the 44pt minimum)
Action-bar card	White, 16pt corners, 12pt padding, 12pt between buttons, shadow 0 – 2 blur 16 #000 at 12%
Overdraw	The drop shadow and the well draw past the bounds. Leave about 3h (78pt) of unclipped room, or set <code>overflow: visible</code>

Layers, from back to front:

1. Page drop shadow and contact shadow (rest only; off by default, strength 0)
2. Well: a soft dark ring round the outline (pressed only; off by default)
3. Face: fill + inner shadows + 2pt border. The pressed face is crossfaded over the rest face
4. Label

Design tokens

The face is a flat fill with two inner shadows and a 2pt gradient border, all taken from Figma. Shadow blur values are given as Figma / CSS blur. In SwiftUI the radius is half the blur. In Skia the sigma is about half the blur.

Face: rest (raised) — Figma, verbatim

Token	Dark	Light
Fill	#212121	#FFFFFF
Border, 2pt inside, linear top → bottom	#EFEFEF → #D2D2D2	#FFFFFF → #DBDFE6
Inner shadow, top edge	#D2D2D2 at 40%, y +4, blur 8	#989FB3 at 10%, y +4, blur 4
Inner shadow, bottom edge	#000000 at 100%, y -9, blur 9	#989FB3 at 22%, y -6, blur 6

Draw the border above the inner shadows, so it stays a crisp ring.

Face: pressed (pushed into the surface) — designed

The pressed face sits below the surface. The surface's edge casts a deep shadow over its top, and a faint light bounces up from the bottom. The face is a shade darker, because less light reaches down there.

Token	Dark	Light
Fill	#1A1A1A	#F3F4F7
Border, 2pt, top → bottom	#2E2E2E → #D2D2D2 (top in shade). Off by default (strokeChange: false): the border keeps its rest colours, so the outline never changes	#C9CED9 → #FFFFFF
Inner shadow 1: edge shade, deep	#000000 at 95%, y +7, blur 14	None. Removed in the Figma dev -> design frame; over the white face it read as a grey band
Inner shadow 2: contact line	#000000 at 70%, y +2, blur 3	#989FB3 at 35% × white top shade 0.6 (= 21%), y +1.5, blur 3
Inner shadow 3: bounce light	#D2D2D2 at 35%, y -3, blur 6	#FFFFFF at 100%, y -3, blur 6
Well (outside the button). Off by default: strength 0, adjustable 0-1	#000000 at 22%: the shape grown 1.5pt, raised 0.75pt, SwiftUI blur radius 1.5	#5B6378 at 18%, same geometry

There is no white top-edge highlight in this state. A lit top edge fights the shade the sunk face is under.

Drop shadow (optional, off by default)

Strength = 0, so the rest state matches Figma, which has no shadow under the button. A strength above 0 adds the shadow below at rest only: it fades to 0 on press, and the light variant uses strength × 0.55. It stays available as a tuning option.

Layer	Colour, dark / light	Offset y	Blur radius	Spread or inset	Opacity
Page shadow	#0E0E0E at 72% / #5B6378 at 55%	+5.1 (0.197h)	3.5 (0.134h)	+1.6 spread (0.06h)	× strength, 0 when pressed

Layer	Colour, dark / light	Offset y	Blur radius	Spread or inset	Opacity
Contact shadow	same as page	+1.95 (0.075h)	1.04 (0.04h)	−0.78 inset (0.03h)	0.9 × strength, 0 when pressed

Typography

Token	Value
Font	Noontree SemiBold (<code>Noontree-SemiBold</code>), style A16/SemiBold
Size / line height	16 / 24pt, tracking 0
Colour	Dark: #FFFFFF (on-surface-bold). Light: #1D2539 (text-n-icon/primary)
Shadow	None. The design's type is flat

States

The button has three states: rest, pressed and disabled. There is no loading or focus state in the design. Pressed begins on **touch-down** (`onPressIn`), not on release.

Property	Rest	Pressed	Disabled
Face	Figma face: glow at top, shade at bottom	Pushed in: deep shade at top, faint bounce light at bottom, fill a shade darker	No surface. The screen draws a flat muted label on a subtle surface
Border	Light at top	Dark at top, light at bottom	None
Drop + contact shadow	None (matches Figma)	0	None
Well	Hidden	Off by default (strength 0)	None
Scale	1.0	0.98	1.0, no press response
Vertical offset	0	+0.5pt (down)	0

Property	Rest	Pressed	Disabled
Label opacity	100%	Dark 90%, light 80%	100%
Haptic	None	Heavy on press, light on release	None

The depth comes from these cues together. The edge of the hole shades its top, the bottom catches a little light, and the button moves down and away from the finger.

Motion

A press runs two animations at the same time, each on its own curve. A **20ms ease-in-out crossfade** switches the light from rest to pressed. A **spring** scales the button to 0.98 and drops it 0.5pt. Both reverse on release.

Track	What animates	Curve	Value
Light change	Pressed face opacity 0→1, drop + contact shadow →0, label opacity 1→0.9 (well 0→strength, default 0)	Ease-in-out	20ms , the same on press and release
Scale	Whole button, centred, 1 → 0.98	Spring	Response 0.2s, damping ratio 0.94 (bounce 0.06)
Depth	Whole button, translateY 0 → +0.5pt	Same spring as scale	0.5pt
Haptic, down	Fires on touch-down	—	iOS Impact Heavy, intensity 1.0
Haptic, up	Fires on release	—	iOS Impact Light, intensity 0.4

The spring in Reanimated `withSpring` terms (mass 1):

$$k = \left(\frac{2\pi}{0.2}\right)^2 \approx 987, \quad c = 2 \cdot 0.94 \cdot \sqrt{k} \approx 59.1$$

Use `{ mass: 1, stiffness: 987, damping: 59.1 }`. At bounce 0.06 the spring settles almost without overshoot, so the press feels firm.

20ms is about 1 frame at 60fps and 2–3 frames at 120Hz, so the light change is close to a snap. Keep it as a timed animation (`withTiming(1, { duration: 20, easing: Easing.inOut(Easing.ease) })`) rather than an instant set. That way the value stays tunable.

React Native implementation

Draw the whole surface, shadows included, in `@shopify/react-native-skia`. Drive it with `react-native-reanimated` shared values. Do not use RN `shadow*` / `elevation`: Android elevation cannot produce a soft, offset, tinted shadow, and plain Views cannot do the radial face, blurs or cut-out rims. Skia makes iOS and Android render the same pixels.

Concern	Library / API
Touch	<code>Pressable</code> with <code>onPressIn</code> / <code>onPressOut</code> (the press state starts on touch-down)
Surface	Skia <code>Canvas</code> placed absolutely behind the label, 78pt larger on every side, <code>pointerEvents="none"</code>
Animation	Reanimated: <code>pressed</code> (0→1, 20ms timing) and <code>scale</code> (spring) shared values, passed straight into Skia props
Inner shadows	Skia <code><Shadow inner dx dy blur color /></code> children on the filled <code>RoundedRect</code> ; <code>blur</code> = Figma <code>blur ÷ 2</code>
Border	A second <code>RoundedRect</code> inset 1pt, <code>style="stroke"</code> , <code>strokeWidth={2}</code> , with a vertical <code>LinearGradient</code>
Well and drop shadow	A <code>RoundedRect</code> with <code><BlurMask style="normal" /></code> in a <code>Group</code> whose <code>opacity</code> is a shared value: <code>pressed</code> for the well, <code>strength × (1 - pressed)</code> for the drop shadow
Haptics	<code>expo-haptics</code> : <code>impactAsync(Heavy)</code> on press-in, <code>impactAsync(Light)</code> on press-out (RN cannot set intensity; Light is the closest match to 0.4)

Props

Prop	Type	Default	Notes
<code>title</code>	string	—	Label text
<code>icon</code>	ReactNode	—	Optional 24pt leading icon, 8pt gap
<code>variant</code>	'dark' 'light'	'dark'	Light uses the light tokens and shadow $\times 0.55$
<code>onPress</code>	() => void	—	Fires on release, inside the bounds
<code>disabled</code>	boolean	false	No surface, no scale, no haptic
<code>width</code>	number '100%'	'100%'	Height is fixed at 52
<code>radius</code>	number	16	

Theme constants

```
export const DomeButtonTokens = {
  height: 52, radius: 16, borderWidth: 2,
  shadowStrength: 0, // optional rest drop shadow (light  $\times 0.55$ );
  0 = Figma exact (default)
  lightChangeMs: 20, // ease-in-out, each way
  pressScale: 0.98,
  pressDepth: 0.5, // pt, translateY while held
  spring: { mass: 1, stiffness: 987, damping: 59.1 }, // response 0.2s,
  bounce 0.06
  pressedLabelOpacity: { dark: 0.9, light: 0.8 },
  topShade: { dark: 1, light: 0.6 }, // 0-1: multiplies the alpha of the
  pressed inner shadows with dy > 0
  well: { strength: 0, grow: 1.5, rise: 0.75, blur: 1.5 }, // off by default
  strokeChange: false, // false: pressed face uses rest.border, so
  the outline never changes
  dark: {
    label: '#FFFFFF', shadow: 'rgba(14,14,14,0.72)', well:
    'rgba(0,0,0,0.22)',
    rest: {
      fill: '#212121', border: ['#EFEFEF', '#D2D2D2'],
      inner: [
        { color: 'rgba(210,210,210,0.4)', dy: 4, blur: 8 },
        { color: 'rgba(0,0,0,1)', dy: -9, blur: 9 },
      ],
    },
  },
  pressed: {
    fill: '#1A1A1A', border: ['#2E2E2E', '#D2D2D2'],
```

```

    inner: [
      { color: 'rgba(0,0,0,0.95)',      dy: 7,  blur: 14 },
      { color: 'rgba(0,0,0,0.7)',       dy: 2,  blur: 3 },
      { color: 'rgba(210,210,210,0.35)', dy: -3, blur: 6 },
    ],
  },
},
light: {
  label: '#1D2539', shadow: 'rgba(91,99,120,0.55)', well:
'rgba(91,99,120,0.18)',
  rest: {
    fill: '#FFFFFF', border: ['#FFFFFF', '#DBDFE6'],
    inner: [
      { color: 'rgba(152,159,179,0.10)', dy: 4,  blur: 4 },
      { color: 'rgba(152,159,179,0.22)', dy: -6, blur: 6 },
    ],
  },
},
pressed: {
  fill: '#F3F4F7', border: ['#C9CED9', '#FFFFFF'],
  inner: [
    { color: 'rgba(152,159,179,0.35)', dy: 1.5, blur: 3 }, // no deep
edge shade on white
    { color: 'rgba(255,255,255,1)',    dy: -3,  blur: 6 },
  ],
},
},
} as const;
// blur values are Figma/CSS blurs: pass blur / 2 to Skia.

```

Reference skeleton

The skeleton below shows the structure and the animation wiring. It is a starting point, not a finished component. Add the contact shadow from the token tables in the same way.

```

import { Pressable, Text } from 'react-native';
import Animated, { useSharedValue, withTiming, withSpring, useAnimatedStyle,
useDerivedValue, Easing } from 'react-native-reanimated';
import { Canvas, Group, RoundedRect, Shadow, LinearGradient, BlurMask, vec,
rrect, rect } from '@shopify/react-native-skia';
import * as Haptics from 'expo-haptics';

const T = DomeButtonTokens;
const H = 52, h = H / 2, PAD = 3 * h, R = 16;
const ease = { duration: T.lightChangeMs, easing: Easing.inOut(Easing.ease)
};

```

```

/** One face: fill + inner shadows + 2pt border. */
function Face({ W, face }) {
  return (
    <>
      {/* Fill inset 1pt so its edge hides under the border: filled to the
edge,
      its antialiased rim shows as a dark hairline outside the border.
*/}
      <RoundedRect x={PAD + 1} y={PAD + 1} width={W - 2} height={H - 2} r={R
- 1} color={face.fill}>
        {face.inner.map((s, i) => (
          <Shadow key={i} inner dx={0} dy={s.dy} blur={s.blur / 2} color=
{s.color} />
        ))}
      </RoundedRect>
      <RoundedRect x={PAD + 1} y={PAD + 1} width={W - 2} height={H - 2} r={R
- 1} style="stroke" strokeWidth={2}>
        <LinearGradient start={vec(0, PAD)} end={vec(0, PAD + H)} colors=
{face.border} />
      </RoundedRect>
    </>
  );
}

```

```

export function DomeButton({ title, onPress, width: W, disabled, variant =
'dark' }) {
  const t = T[variant];
  const pressed = useSharedValue(0);
  const scale = useSharedValue(1);
  const depth = useSharedValue(0);
  const strength = T.shadowStrength * (variant === 'light' ? 0.55 : 1);
  const lift = useDerivedValue(() => strength * (1 - pressed.value));
  const well = useDerivedValue(() => pressed.value * T.well.strength);

  const down = () => {
    if (disabled) return;
    pressed.value = withTiming(1, ease);
    scale.value = withSpring(T.pressScale, T.spring);
    depth.value = withSpring(T.pressDepth, T.spring);
    Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Heavy);
  };
  const up = () => {
    if (disabled) return;
    pressed.value = withTiming(0, ease);
    scale.value = withSpring(1, T.spring);
    depth.value = withSpring(0, T.spring);
  };
}

```

```

    Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Light);
  };

  const box = useAnimatedStyle(() => ({ transform: [{ translateY: depth.value
}, { scale: scale.value }] }));
  const label = useAnimatedStyle(() => ({ opacity: 1 - pressed.value * (1 -
T.pressedLabelOpacity[variant]) }));
  const g = T.well.grow;

  return (
    <Pressable onPressIn={down} onPressOut={up} onPress={onPress} disabled=
{disabled}
      accessibilityRole="button" accessibilityLabel={title}
      accessibilityState={{ disabled }}>
      <Animated.View style={[{ width: W, height: H, alignItems: 'center',
justifyContent: 'center' }, box]>
        {!disabled && (
          <Canvas pointerEvents="none" style={{ position: 'absolute', left: -
PAD, top: -PAD, width: W + 2 * PAD, height: H + 2 * PAD }}>
            { /* Rest drop shadow: fades out on press */ }
            <Group opacity={lift} transform={[{ translateY: 0.197 * h }]}>
              <RoundedRect rect={rrect(rect(PAD - 0.06 * h, PAD - 0.06 * h, W
+ 0.12 * h, H + 0.12 * h), R, R)} color={t.shadow}>
                <BlurMask blur={0.134 * h} style="normal" />
              </RoundedRect>
            </Group>
            { /* Well: off by default (strength 0) */ }
            <Group opacity={well} transform={[{ translateY: -T.well.rise }]}>
              <RoundedRect rect={rrect(rect(PAD - g, PAD - g, W + 2 * g, H +
2 * g), R + g, R + g)} color={t.well}>
                <BlurMask blur={T.well.blur} style="normal" />
              </RoundedRect>
            </Group>
            <Face W={W} face={t.rest} />
            <Group opacity={pressed}>
              <Face W={W} face={T.strokeChange ? t.pressed : { ...t.pressed,
border: t.rest.border }} />
            </Group>
            { /* + contact shadow, opacity 0.9 × lift */ }
          </Canvas>
        )}
      <Animated.View style={label}>
        <Text style={{ fontFamily: 'Noontree-SemiBold', fontSize: 16,
lineHeight: 24, color: t.label }}>{title}</Text>
      </Animated.View>
    </Pressable>
  );

```

```

    </Animated.View>
  </Pressable>
);
}

```

A note on porting blurs: SwiftUI `.blur(radius:)` and Skia `blur` (sigma) are not guaranteed to match exactly. Start 1:1 with the values in this spec, then compare screenshots side by side against the iOS build. Run the iOS build with the `-pressHeld` launch argument to freeze the pressed look for comparison.

Accessibility and QA

The button meets the 44pt touch minimum at 52pt. It must announce itself as a button with its label and disabled state. White on the dark face and #1D2539 on the light face both clear 4.5:1 contrast.

- ☐ `accessibilityRole="button"`, `accessibilityLabel` = the title, `accessibilityState={{ disabled }}`
- ☐ Reduce Motion on: skip the 0.98 scale spring and the 0.5pt drop; keep the 20ms light change and the haptics
- ☐ Haptics respect the app-wide haptics setting
- ☐ Disabled: no surface, no scale, no haptic, `onPress` never fires
- ☐ The press look starts on touch-down, not on release
- ☐ Dragging off the button while held releases the press look; `onPress` does not fire
- ☐ Shadows and rim are not clipped by a parent (`overflow: visible`, or room for the 78pt overdraw)
- ☐ Side-by-side screenshots against the iOS build, rest and pressed, for both variants, on iOS and Android
- ☐ The label font is Noontree SemiBold 16/24 on both platforms (font is linked and loaded)
- ☐ Holds 60fps while pressing rapidly (shared values only, no React re-render per press)